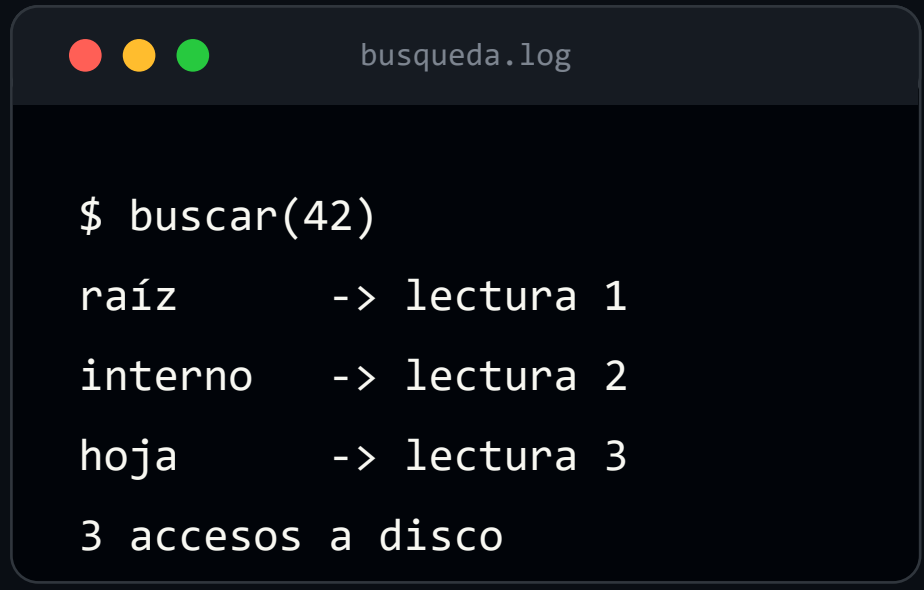
CE-2103 · Lección 06

Almacenamiento Externo

Estructuras de datos para el disco



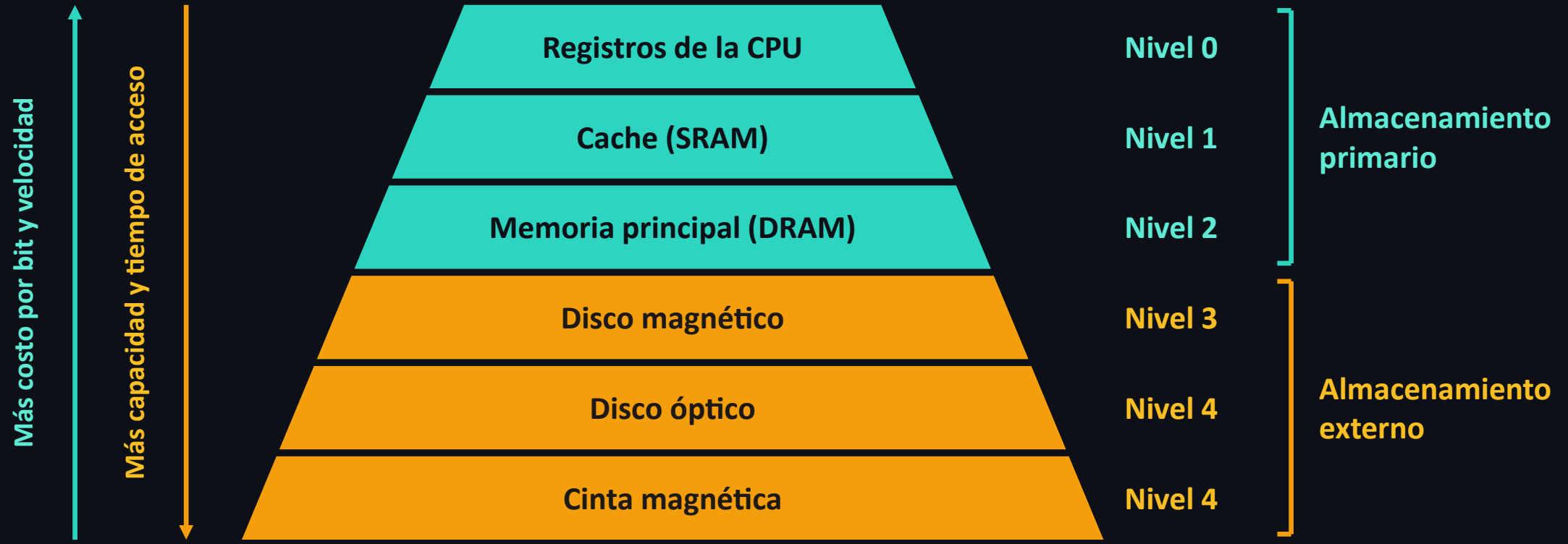
Instituto Tecnológico de Costa Rica · Ingeniería en Computadores

¿Qué aprenderemos hoy?

1. Describir la **jerarquía de memoria** y comparar **HDD** y **SSD**
2. Explicar **RAID** y reconstruir un disco con **paridad (XOR)**
3. Explicar **particiones**, **sistemas de archivos**, **archivos** e **i-nodos**
4. Aplicar el **cache** y sus políticas de reemplazo (LRU, FIFO…)
5. Ordenar datos que no caben en memoria: **external merge** y **merge-sort**
6. Explicar por qué los **árboles B y B+** minimizan los accesos a disco
7. Insertar claves en un árbol B (**split**) y comparar B con B+

// jerarquía de memoria

La jerarquía de memoria



Arriba: poca capacidad, muy rápida y cara. Abajo: gran capacidad, más lenta y barata.

// conceptos esenciales

Dato, información y almacenamiento

Dato

Sucesión de símbolos (números o letras) que representa un hecho. Por sí solo no informa.

$10 · 20 años · 3 kg

Información

Interpretación significativa de los datos. Requiere organizarlos y procesarlos.

Datos + interpretación

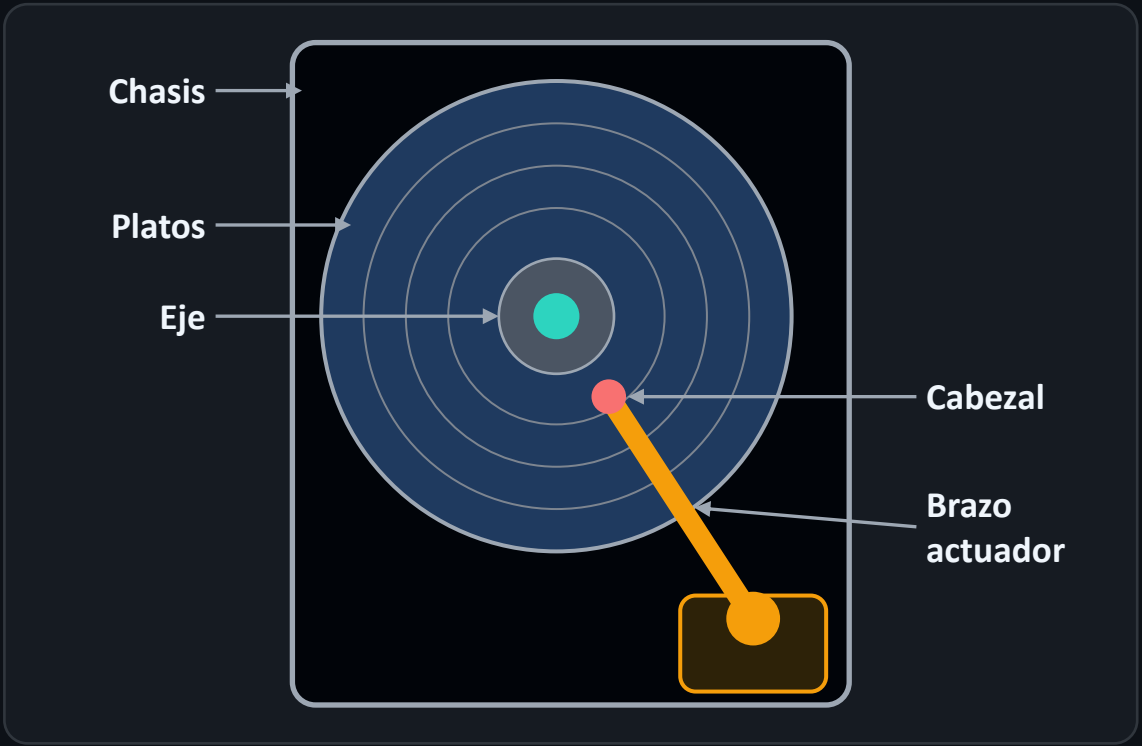
Almacenamiento

Sistemas que guardan datos en formato binario, sin importar qué información contengan.

Solo ven bloques de bits

Para un sistema de almacenamiento todo son **bloques crudos de bits**: el significado lo dan las capas superiores.

Componentes de un disco duro

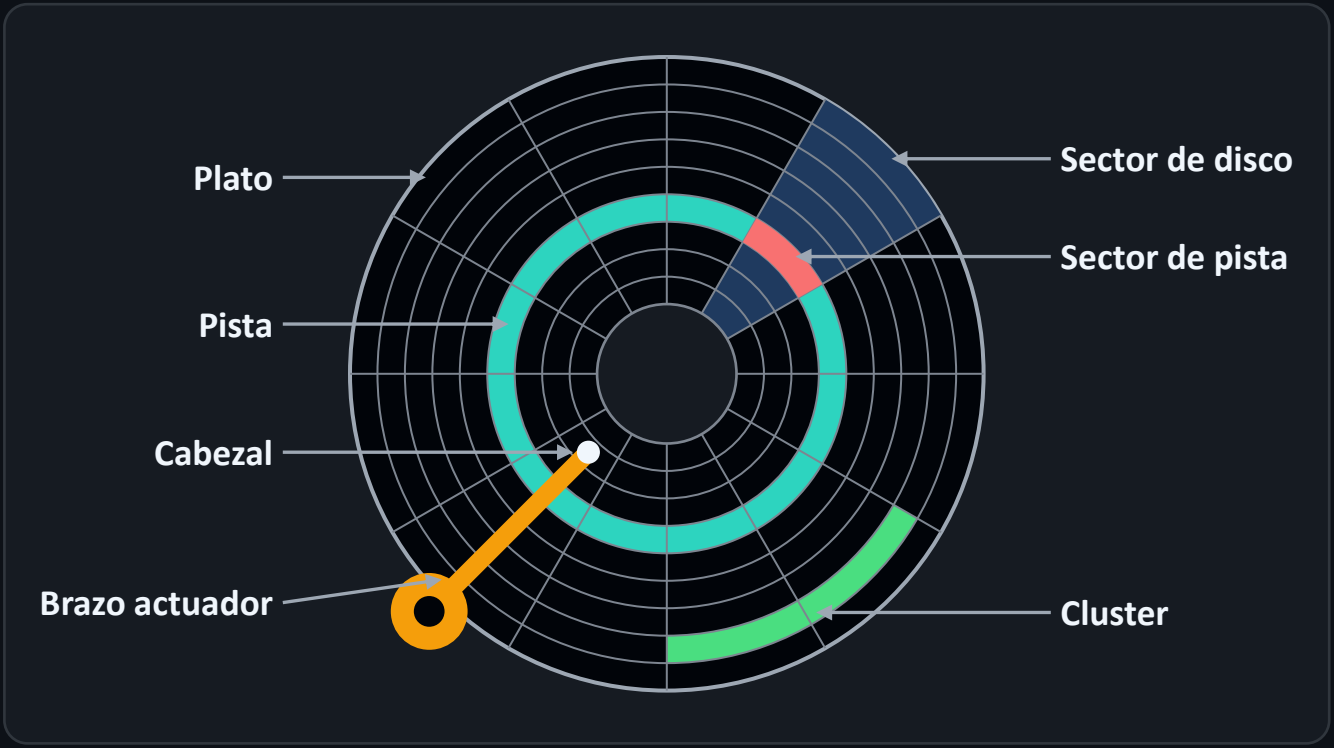


- **Platos**
  Discos con material magnético en ambas caras.
- **Eje**
  Sostiene los platos y los gira a RPM constante.
- **Brazo actuador**
  Mueve los cabezales a lo largo de los platos.
- **Cabezales**
  Leen y escriben los datos en los platos.

Giro: 5400 / 7200 RPM (consumidor) y 10 000 / 15 000 RPM (servidores). **Más RPM** → más velocidad de E/S, pero más consumo y costo.

// discos duros (hdd)

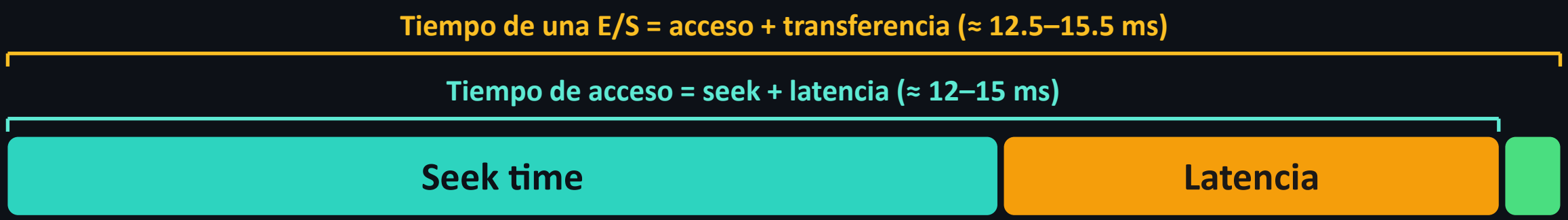
Pistas y sectores



- **Pistas**
  Círculos concéntricos en los que se escriben los datos.
- **Sectores**
  Cada pista se divide en sectores.
- **Unidad mínima**
  Un sector (generalmente 512 bytes) es la unidad mínima de almacenamiento.

El disco no lee ni escribe bytes sueltos: trabaja con **sectores completos**.

Tiempos de acceso y transferencia



Seek time

Tiempo que tarda el brazo actuador en moverse a la pista deseada.

8–10 ms (7200 RPM)

Latencia

Tiempo que tarda el sector deseado en pasar bajo el cabezal.

4–5 ms (7200 RPM)

Transferencia

Tiempo de leer o escribir el bloque, según su tamaño y la tasa de transferencia.

≈ 0.5 ms (ejemplo)

Transferencia = tamaño del bloque / tasa de transferencia. Con un external data rate de 300 MB/s se transfieren 300 MB por segundo.

Interfaces de un disco duro

Interfaz	Velocidad	Hot-swappable	Uso
PATA	133 MB/s	No	Solo discos internos; ya no se usa
SATA	600 MB/s	Sí	Estándar actual; bajo consumo
SCSI	Alta velocidad	Sí	Servidores y estaciones de trabajo
SAS	12 Gb/s (≈ 1.5 GB/s)	Sí	Servidores y estaciones de trabajo

SATA evolucionó de **PATA**; **SAS** (Serial Attached SCSI) evolucionó de **SCSI**.

// discos de estado sólido (ssd)

HDD vs. SSD

Disco duro (HDD)

- Platos magnéticos con partes móviles
- Paga seek time y latencia
- Acceso aleatorio lento
- Menor precio

vs

Estado sólido (SSD)

- Memoria flash, sin partes móviles
- Sin latencia de búsqueda
- Acceso aleatorio mucho más rápido
- Writes limitados: menor vida útil
- Precio más alto

Un SSD **no** es un disco duro: no tiene partes móviles, por eso es más rápido y menos propenso a fallas.

// discos de estado sólido (ssd)

Componentes de un SSD

Controlador

Procesador que ejecuta el firmware: gestiona la memoria y corrige errores.

Memoria NAND

Memoria flash que guarda los datos, en celdas SLC, MLC, TLC o QLC.

DRAM

Memoria volátil para datos temporales y para mejorar el rendimiento.

Firmware

Software que controla el funcionamiento del SSD.

Conector

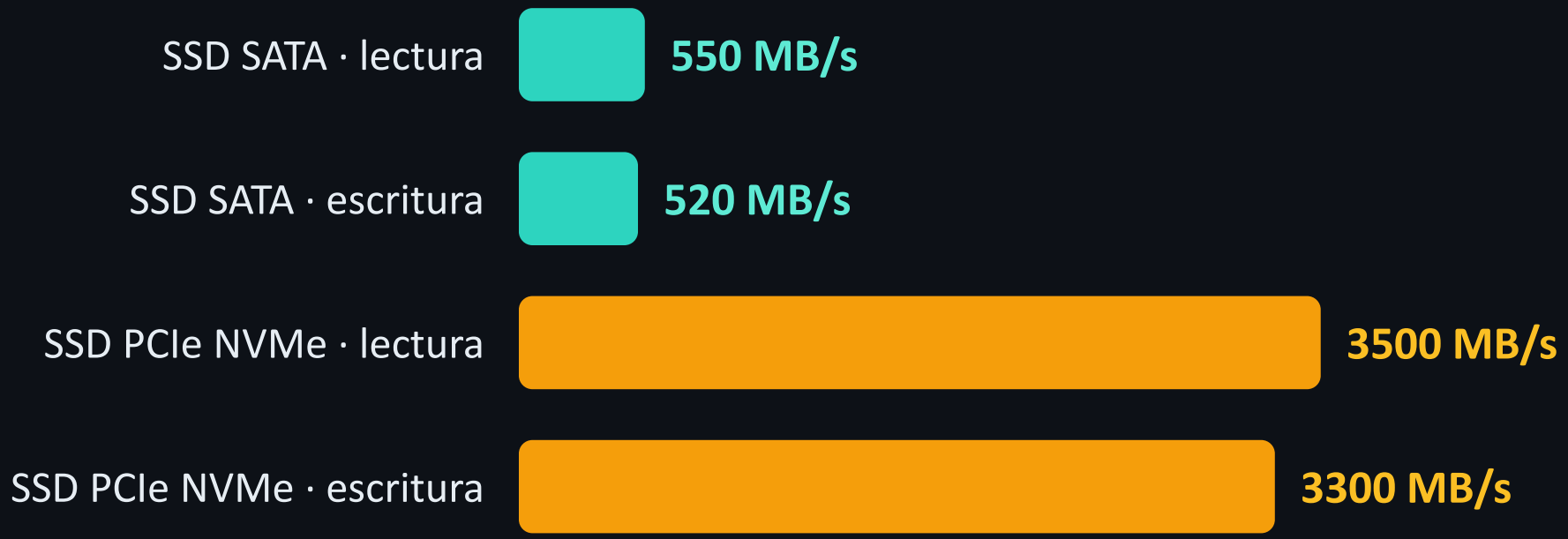
Une el SSD a la placa madre: SATA o PCIe (protocolo NVMe).

Form factor

Tamaño y forma: 2.5", M.2, U.2 o tarjeta de expansión PCIe.

// discos de estado sólido (ssd)

Desempeño: velocidades máximas



La transferencia de un SSD es mucho más rápida que la de un disco duro; con **NVMe** la lectura es ≈ 6 veces mayor que con SATA.

// arreglos de discos (raid)

RAID: varios discos que actúan como uno

Striping



Divide los datos secuenciales entre varios discos que trabajan en paralelo.

RAID 0 · más rendimiento

Mirroring



Réplica idéntica en otro disco: el servicio continúa si uno falla.

RAID 1 · disponibilidad

Paridad



PA se calcula con A1 y A2: si un disco falla, su bloque se reconstruye.

RAID 5 y 6 · tolera fallos

Los niveles de **RAID** combinan estas técnicas: **striping** para el rendimiento; **mirroring** y **paridad** para tolerar fallos.

Niveles de RAID más usados

Nivel	Técnica	Mínimo	Tolera	Capacidad útil	Se destaca por
RAID 0	Striping por bloques	2 discos	Ningún disco	N × D	Rendimiento
RAID 1	Mirroring (espejo)	2 discos	1 disco	D	Redundancia
RAID 5	Striping + paridad distribuida	3 discos	1 disco	(N − 1) × D	Lecturas rápidas
RAID 6	Striping + doble paridad distribuida	4 discos	2 discos	(N − 2) × D	Mayor seguridad
RAID 10	Franjas de espejos (1+0)	4 discos	1 por espejo	N / 2 × D	Rendimiento y redundancia

Ejemplo: con N = 4 discos de D = 1 TB, la capacidad útil es **4 TB** en RAID 0, **3 TB** en RAID 5 y **2 TB** en RAID 6 y RAID 10.

El bit de paridad

Datos (8 bits)	N.º de unos	Con paridad par	Con paridad impar
11001100	4	11001100 **0**	11001100 **1**
11111011	7	11111011 **1**	11111011 **0**
11001111	6	11001111 **0**	11001111 **1**
01111111	7	01111111 **1**	01111111 **0**

Paridad par: el bit extra es **1** si la cantidad de unos es **impar**; así el total queda par.

Paridad impar: el bit extra es **1** si la cantidad de unos es **par**; así el total queda impar.

Un bit de paridad **detecta** que un bit cambió, pero no dice **cuál**. En RAID, el mismo truco (XOR) permite **reconstruir** un disco completo.

RAID 5: reconstruir un disco con XOR

	Disco 1	Disco 2 (falló)	Disco 3	Disco 4
Franja 1	0100	????	0010	P 0011
Franja 2	0010	????	P 0110	0100
Franja 3	0011	????	1010	1000
Franja 4	P 0110	????	1101	1010

P = bloque de paridad; rota entre los discos de una franja a otra.

Reconstruir la franja 1

Se aplica XOR a los bloques que sobreviven de esa franja:

```
0100 XOR 0010 = 0110
0110 XOR 0011 = 0101
```

Disco 2, franja 1 = 0101

XOR da 1 si los dos bits son distintos.

Con un disco caído el arreglo **sigue funcionando** (modo degradado: más lento, reconstruye al vuelo). **RAID 5 tolera un solo disco.**

Particiones y volúmenes

Un solo disco físico



Tabla de particiones

Metadata con el tamaño, la ubicación y el tipo de sistema de archivos de cada partición.

Partición

Se comporta como un disco lógico independiente. Puede existir sin inicializarse.

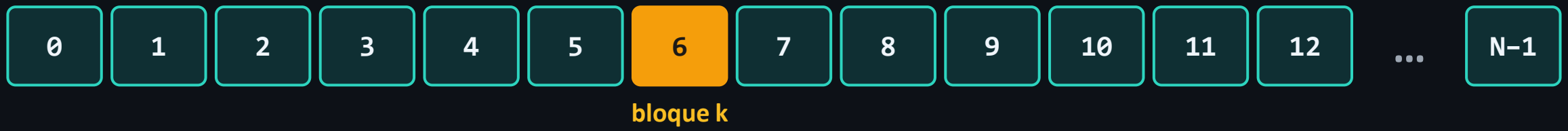
Volumen

Partición inicializada con un sistema de archivos: la interfaz lógica que usa el SO.

> **MBR**: la tabla está en el primer sector del disco. **GPT** (GUID Partition Table): esquema moderno, con copias redundantes y sin las limitaciones de MBR.

Del disco al sistema de archivos

Desde aquí, el sistema operativo ve el disco como una **secuencia lineal de bloques**:



Solo dos operaciones: **leer el bloque k** **escribir el bloque k**

Encontrar

¿Cómo encontramos la información?

→ **Nombres y directorios**

Proteger

¿Cómo evitamos que un usuario lea los datos de otro?

→ **Permisos**

Administrar

¿Cómo sabemos qué bloques están libres?

→ **Espacio libre**

El **sistema de archivos** responde las tres preguntas.

Sistema de archivos

Abstracción

Interfaz lógica del SO entre el usuario y la capa física de almacenamiento.

Nombra y organiza

Metadata

Guarda atributos de cada archivo: nombre, tamaño, fecha de creación…

Ocupa espacio en disco

Según el SO

Windows usa NTFS; Linux, ext3 o ext4; macOS, APFS (antes HFS+).

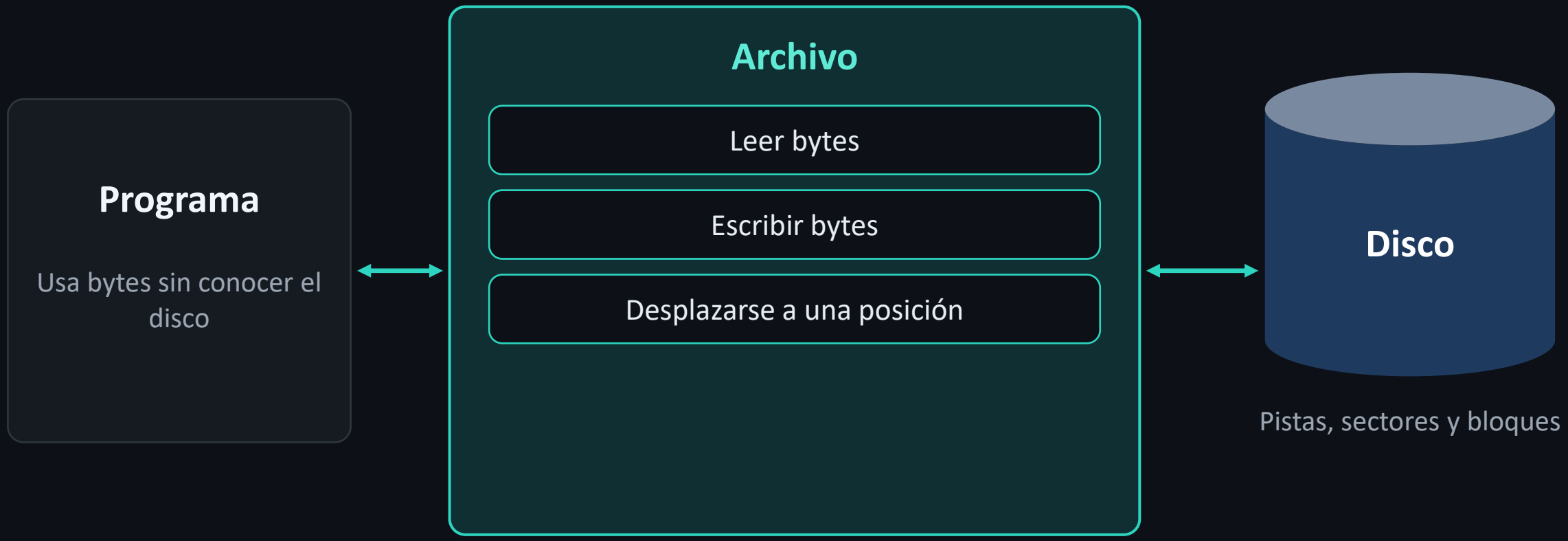
NTFS · ext4 · APFS

Tras **formatear** un disco, el espacio disponible es menor al total: el sistema de archivos necesita espacio para su metadata.

¿Para qué sirve un sistema de archivos?

- **Nombrar y organizar** los archivos
- Ofrecer un **API de acceso**: crear, leer, escribir, eliminar…
- Mantener un **índice** de archivos y directorios
- Gestionar **permisos y restricciones** para la integridad y la seguridad
- **Funciones avanzadas**: compresión, cifrado, cuotas de disco

El archivo: una abstracción del disco



Un archivo es un «canal de comunicación» que oculta al usuario los detalles complejos del disco.

Formas de acceder a un archivo

Secuencial

Byte por byte, desde el principio hasta el final, en el orden en que se escribieron.

Más rápido

Aleatorio

Acceso directo a cualquier parte del archivo, sin importar el orden de escritura.

Más flexible

Indexado

Un índice guarda la ubicación de los datos, p. ej. id de cliente → posición del registro.

Precursor de las BD

> El acceso secuencial es más rápido; el aleatorio es más flexible. El indexado depende de la estructura y el propósito del archivo.

Estructura interna de un archivo

Secuencia de bytes

Archivo plano sin estructura interna: el SO no conoce su contenido. Máxima flexibilidad.

Windows · Linux · macOS

Registros de tamaño fijo

Campos de tamaño fijo definidos al crear el archivo. Permite buscar y ordenar.

Mainframes y BD

Árboles de registros

Cada nodo guarda un registro y apunta a otros nodos. Búsqueda y orden eficientes.

Idea de los árboles B/B+

Cuanta más estructura conoce el sistema, más operaciones puede ofrecer: buscar, ordenar e indexar.

// archivos

Tipos de archivos

Regulares

Datos de usuario; de texto o binarios.

Directorios

Contienen la lista de archivos y directorios.

Character special

Dispositivos de caracteres: terminales y puertos serie.

Block special

Dispositivos de bloques: discos duros y unidades flash.

Sockets

Permiten la comunicación entre procesos.

Pipes

Permiten la comunicación entre procesos.

Symbolic links

Archivos que apuntan a otros archivos.

Binary files

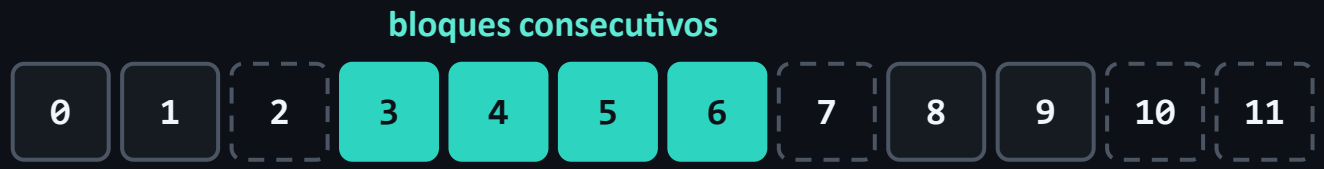
Binarios: ejecutables, imágenes, video.

¿Qué bloques pertenecen a cada archivo?

bloques del archivo A bloques de otros archivos bloque libre

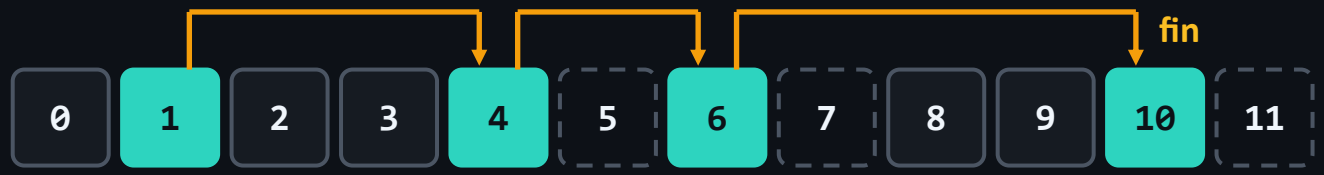
Contigua

Los bloques del archivo son vecinos. Lectura rápida, pero desfragmentar el disco es carísimo.



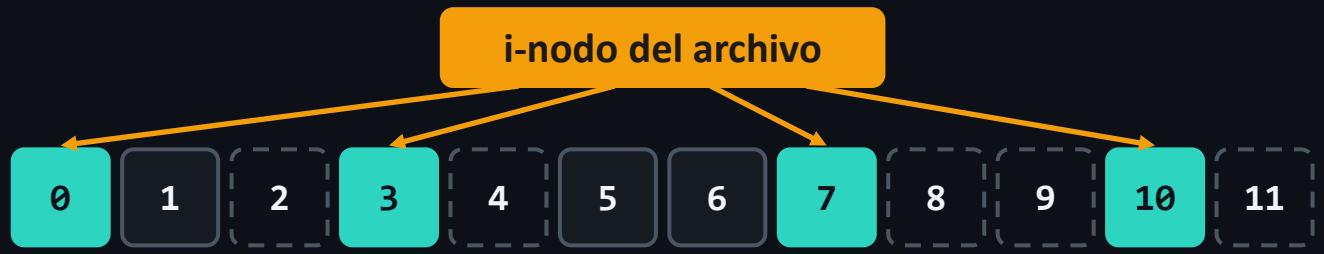
Lista enlazada

Cada bloque apunta al siguiente. El acceso aleatorio es doloroso y los punteros están en el disco.



I-nodo

Un i-nodo por archivo: atributos y direcciones de sus bloques. Está en memoria solo si el archivo está abierto.

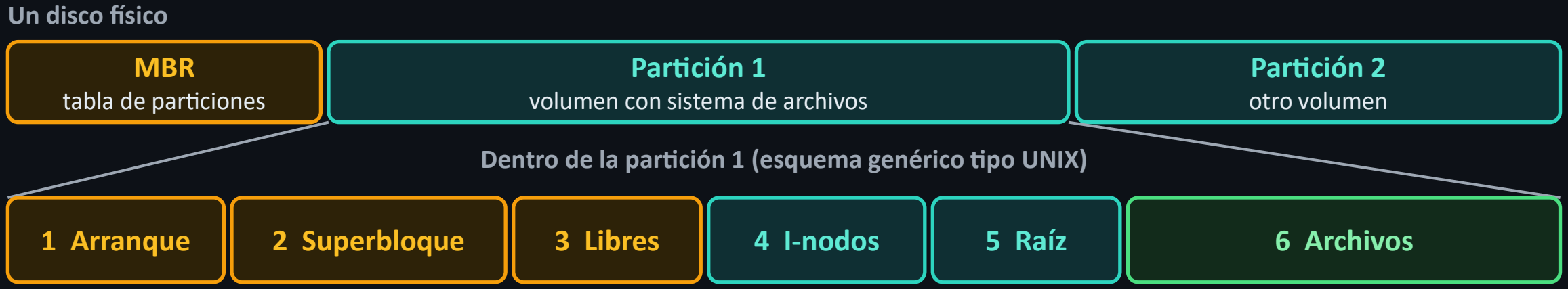


I-nodo: 12 directos y 3 indirectos

Puntero del i-nodo	Camino hasta los datos	Bloques de datos	Tamaño
Directos	1–12 → datos	12	6 KiB
Indirecto simple	13 → punteros → datos	128	64 KiB
Indirecto doble	14 → punteros → punteros → datos	$128^2 = 16\,384$	8 MiB
Indirecto triple	15 → punteros → punteros → punteros → datos	$128^3 = 2\,097\,152$	1 GiB
Total	$12 + 128 + 128^2 + 128^3$	2 113 676	≈ 1032 MiB

Con bloques de **512 B** y punteros de **4 B**, un bloque de punteros guarda 512 / 4 = **128** punteros.

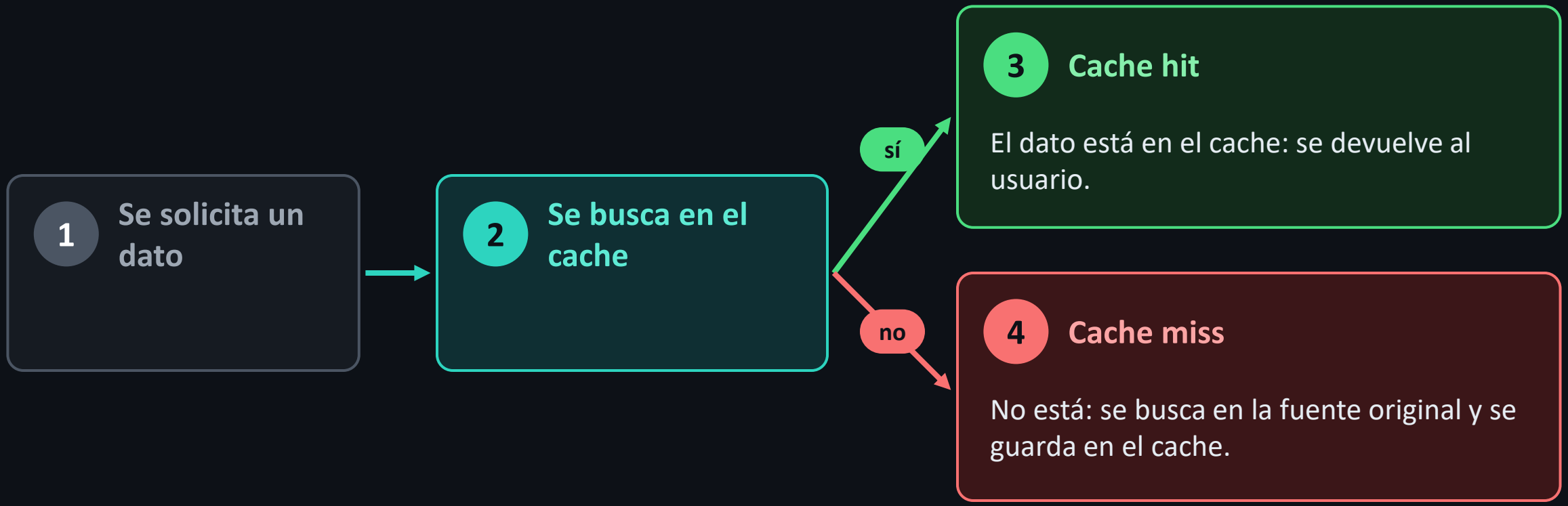
Anatomía de un disco con sistema de archivos



1. **Bloque de arranque:** programa pequeño que carga el sistema operativo desde la partición
2. **Superbloque:** parámetros clave del sistema de archivos, p. ej. cuántos bloques tiene
3. **Espacio libre:** registro de qué bloques están libres
4. **I-nodos:** uno por archivo: sus atributos y las direcciones de sus bloques
5. **Directorio raíz:** origen del árbol de directorios
6. **Archivos y directorios:** los bloques de datos de todo lo demás

Bloque del sistema de archivos ≥ sector: el SO lee y escribe de a bloques, que son múltiplos del sector. Es un esquema genérico: ext4 o NTFS lo organizan distinto.

Cómo funciona un cache



Principio de localidad: los programas acceden a un conjunto reducido de direcciones en un periodo corto; conviene tenerlas cerca.

Políticas de reemplazo

Cuando el cache está lleno, cada miss obliga a elegir qué entrada sacar:

LRU

Reemplaza la entrada que no se ha usado por más tiempo.

Least Recently Used

FIFO

Reemplaza la entrada que lleva más tiempo en el cache. La más simple.

First In First Out

LFU

Reemplaza la entrada que se ha usado menos veces.

Least Frequently Used

MRU

Reemplaza la entrada que se ha usado más recientemente.

Most Recently Used

Todas responden la misma pregunta: **¿qué entrada sacamos?** LRU es la política más común.

Ejemplo: LRU vs. FIFO con 3 entradas



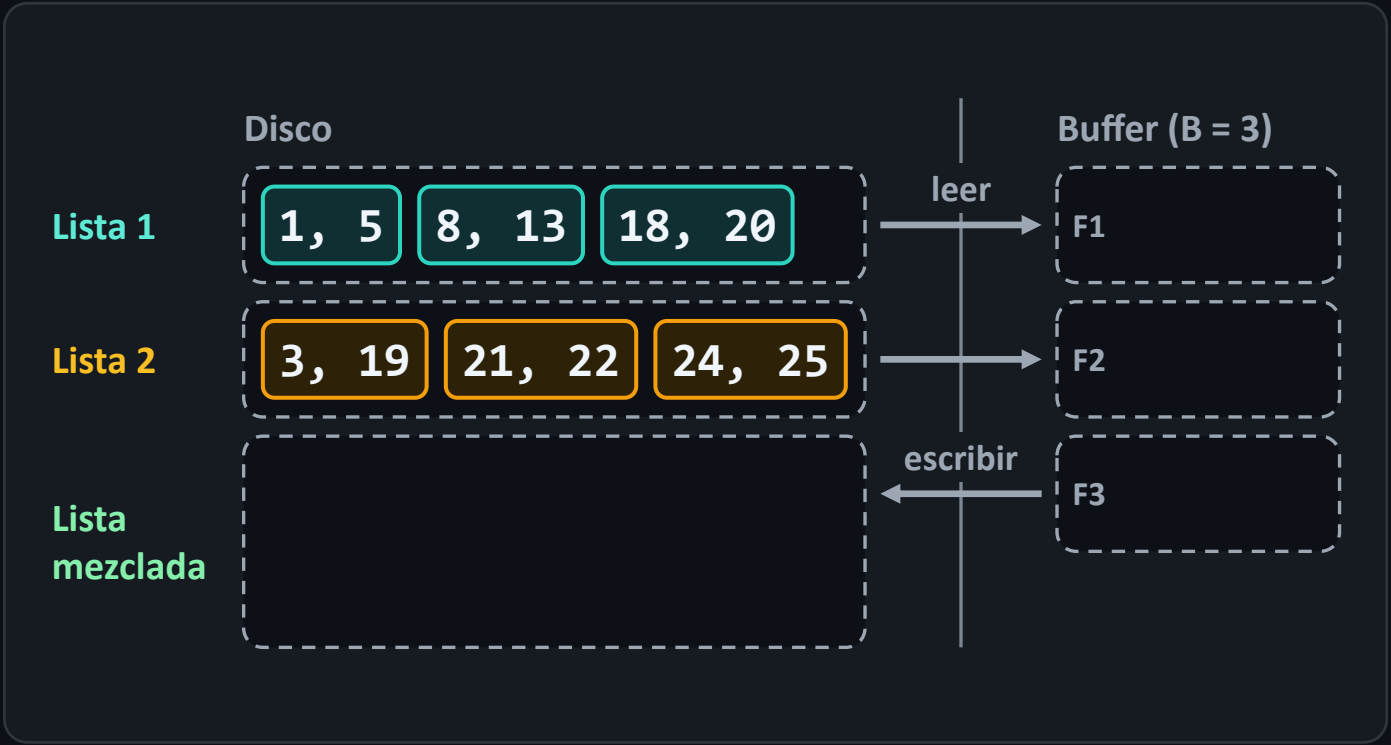
LRU logra **2 hits** y FIFO solo **1**: FIFO expulsa a **A** por ser la más antigua, aunque se acaba de usar.

Tipos de cache

Tipo	Dónde se almacena	Se usa para…
En memoria	RAM	Datos frecuentes del proceso
De disco	Disco duro	Datos frecuentes de un archivo o del disco
Distribuido	Varios servidores	Datos frecuentes en la red (un sitio web)
Application server cache	RAM del servidor de aplicaciones	Datos frecuentes de la aplicación
Global	Varios servidores	Datos frecuentes a nivel global

> **Generales:** en memoria, de disco y distribuido. **Específicos:** application server cache (una especialización del cache en memoria) y global.

External merge: unir dos archivos ordenados

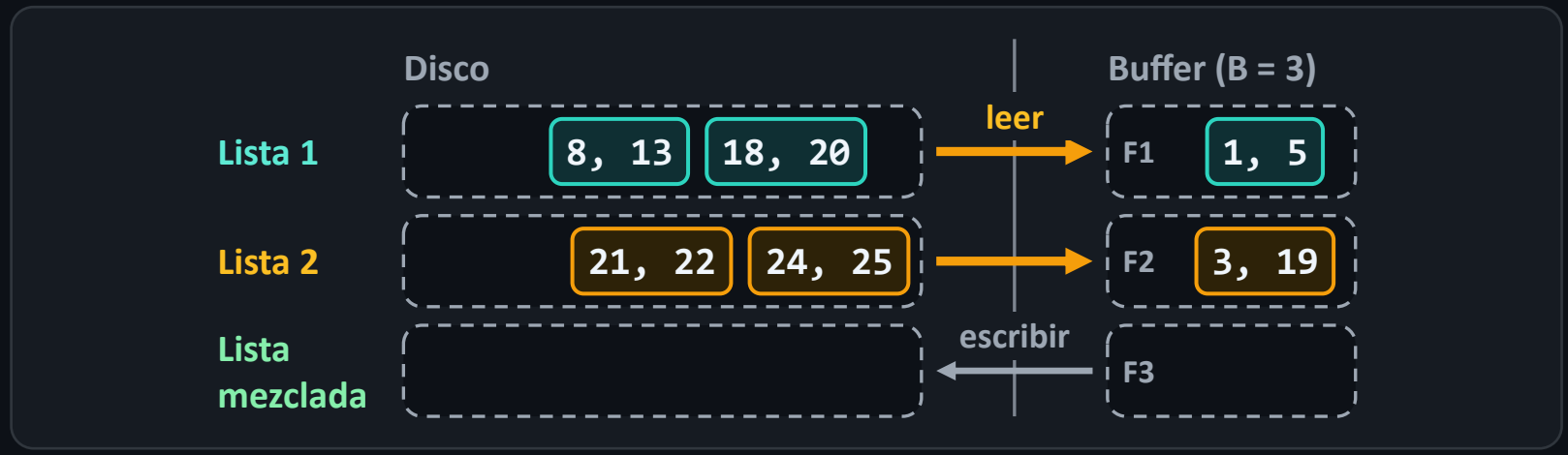


- **Dos archivos ordenados**
  No caben en memoria: se procesan por bloques de tamaño n.
- **Tres frames en memoria**
  Uno por archivo de entrada y uno temporal de salida.

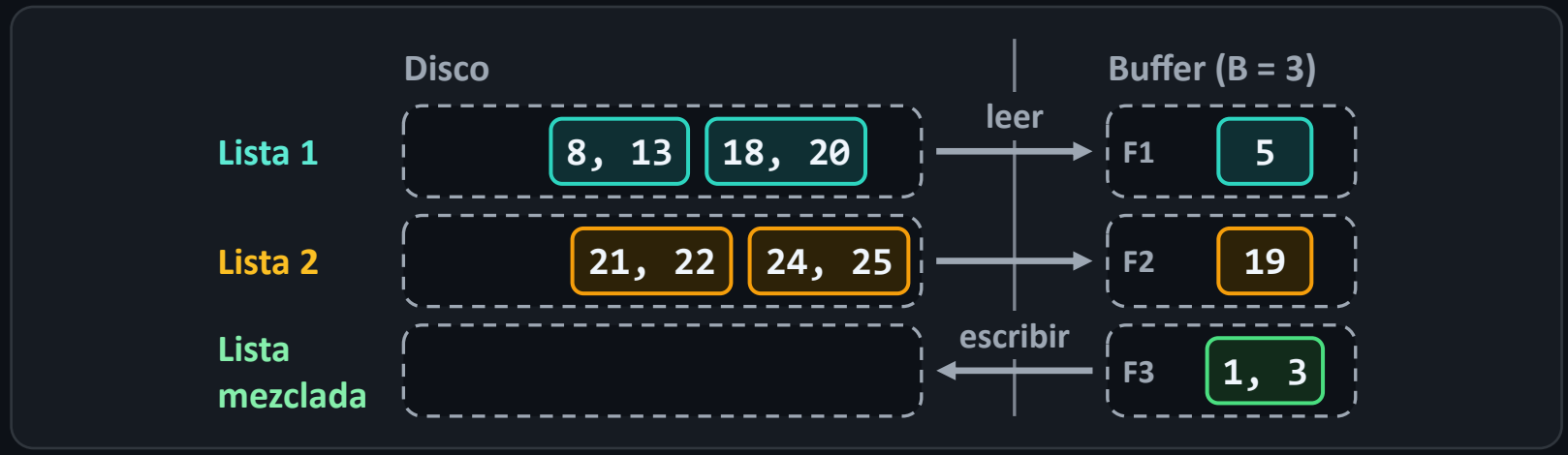
Los algoritmos de **ordenamiento externo** usan un enfoque híbrido: memoria principal y memoria secundaria.

External merge: leer y mezclar

1. Se lee la primera página de cada lista en un frame de entrada.



2. Se mezclan en el frame temporal hasta que se llena (aquí: 1, 3).

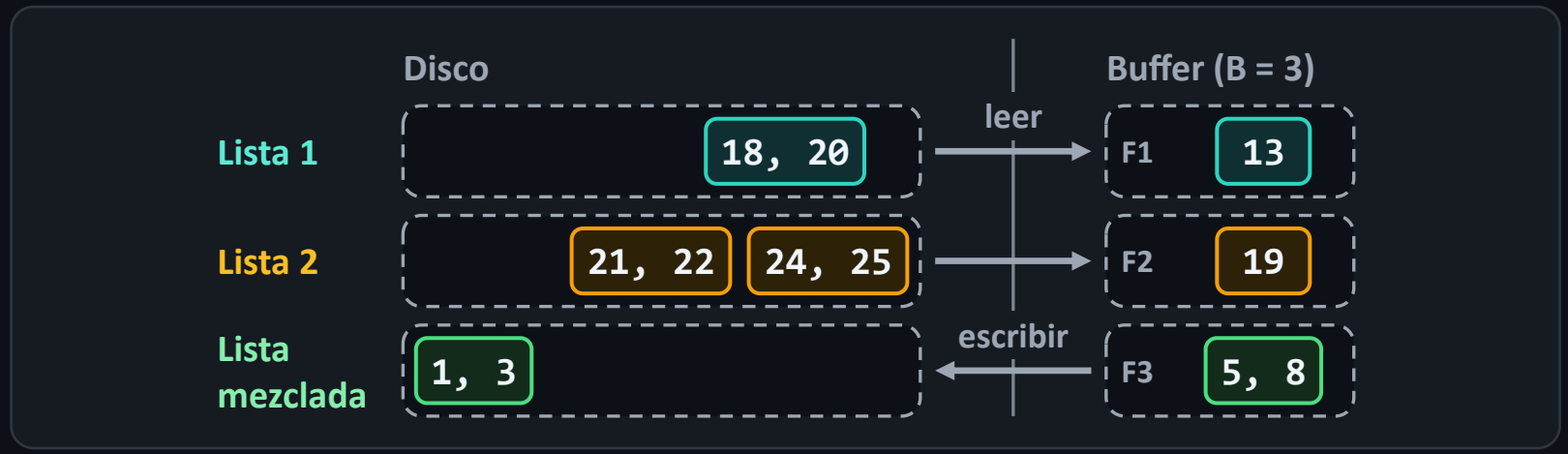


External merge: escribir y continuar

3 La página llena se escribe en el archivo de salida.



4 Se sigue mezclando; cuando un frame de entrada se vacía, se lee otra página.



External merge-sort: runs y etapas

Run

Archivo temporal ordenado en disco que hace de «subarray». Un run de tamaño 1 es una página.

Subarray en disco

Etapa de división

Se lee página por página; cada una se ordena y se escribe como un run de tamaño 1.

n páginas → n runs

Etapa de mezcla

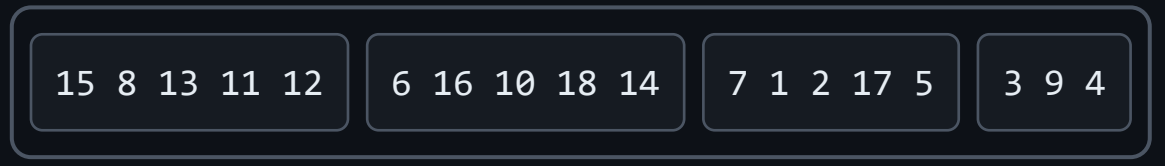
Se mezclan dos runs de tamaño i con external merge y se obtiene un run de tamaño 2·i.

i + i → 2·i

En memoria solo hay **3 frames**: el archivo se ordena con pasadas sucesivas sobre el disco.

Ejemplo: 18 elementos en 4 páginas

Archivo original
4 páginas · 18 elementos



División
4 runs de 1 página



Mezcla 1
2 runs de 2 páginas

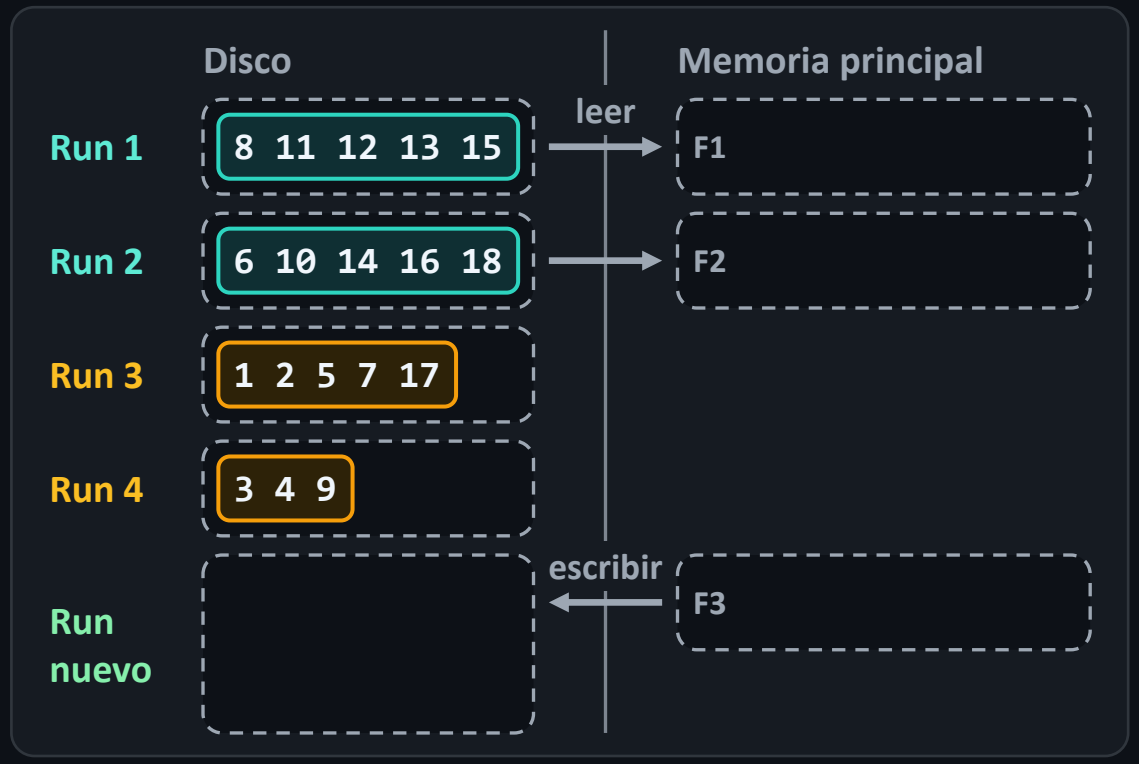


Mezcla 2
1 run de 4 páginas: ordenado

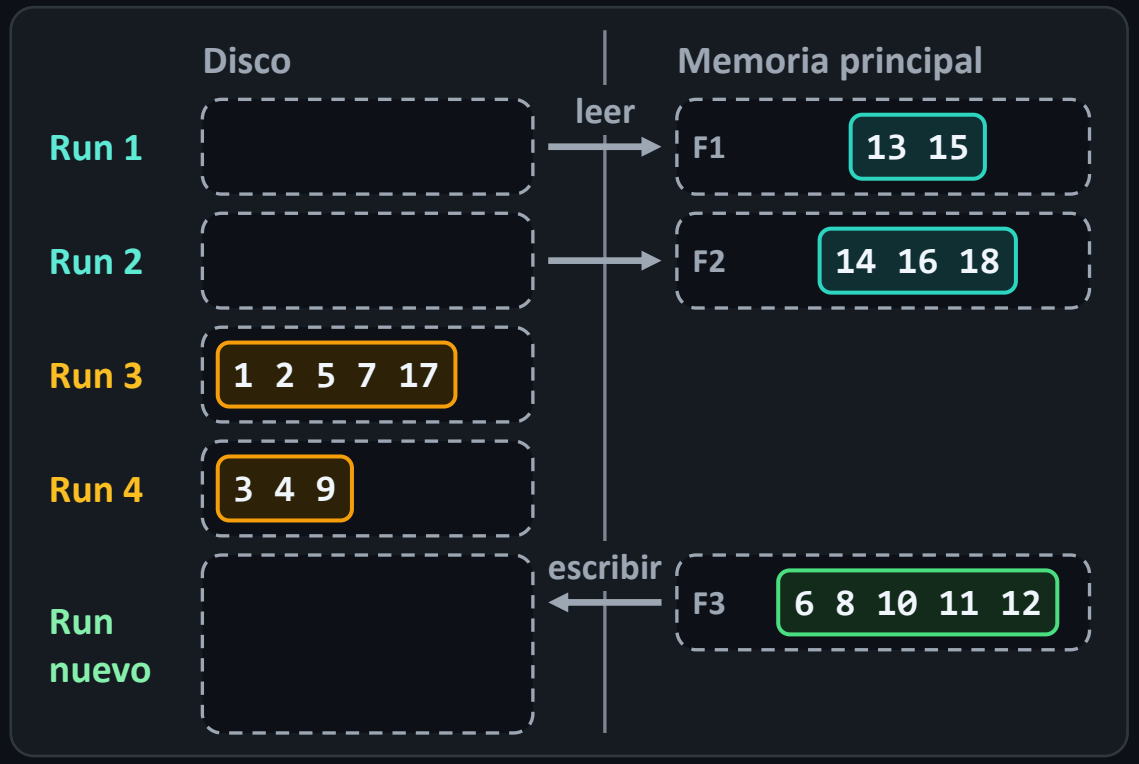


Cada mezcla duplica el tamaño de los runs (4 → 2 → 1). Con **multiway merge** se reduce el número de pasadas.

Ejemplo: división y mezcla de dos runs



1 Tras la división: 4 runs de tamaño 1 en disco y 3 frames vacíos en memoria.



2 Mezcla de los dos primeros runs: F1 y F2 alimentan a F3, que escribirá un run nuevo de tamaño 2.

Buscar: en memoria vs. en disco

En memoria

- Costo dominante: las **comparaciones**
- Una comparación toma **nanosegundos**
- BST balanceado: búsqueda en O(log n)

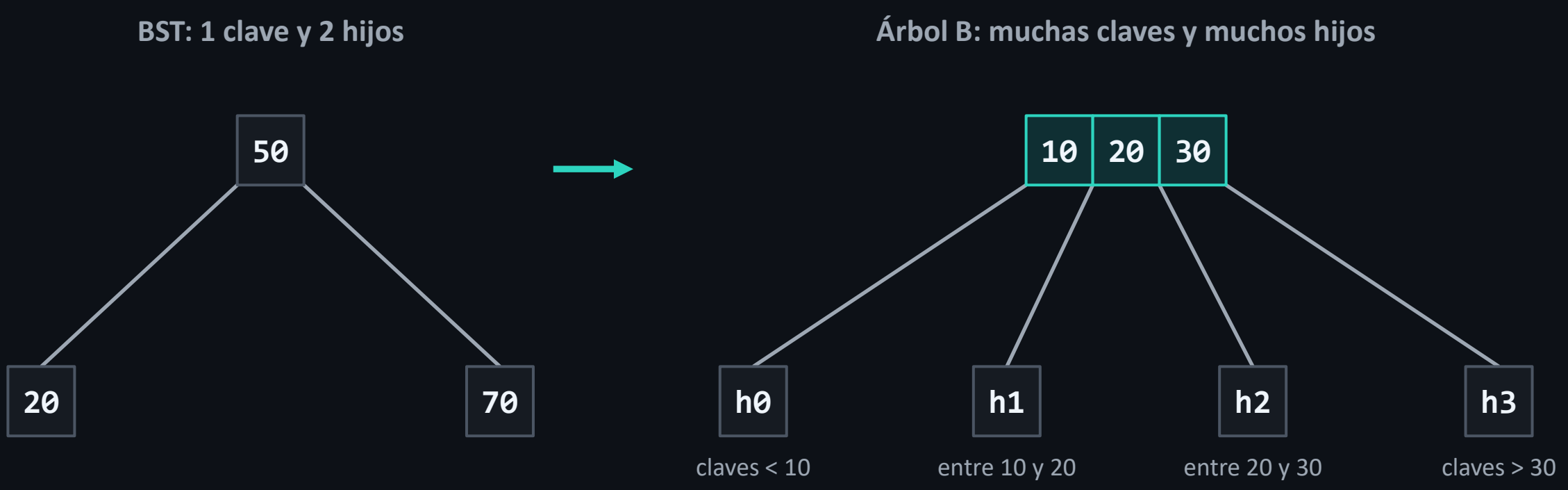
vs

En disco

- Costo dominante: la **E/S** (I/O)
- Cada acceso: seek + latencia, en **milisegundos**
- Una lectura puede costar millones de comparaciones

Un BST en disco: con 1 000 000 de elementos, $\log_2(1\,000\,000) \approx$ **20 accesos** por búsqueda. Inaceptable.

Idea clave: nodos «gordos»



Regla de diseño: un nodo = una página de disco. Una lectura trae un nodo completo y cada acceso decide entre muchos hijos, no solo entre dos.

Árbol B de grado mínimo t (t ≥ 2)

- Todo nodo, salvo la raíz, tiene **al menos t – 1 claves** (y al menos t hijos)
- Todo nodo tiene **a lo sumo 2t – 1 claves** (2t hijos); con 2t − 1 está **lleno**
- La **raíz** tiene al menos 1 clave
- Las claves de cada nodo están **ordenadas**
- **Todas las hojas** están a la misma profundidad

Altura y costo de una búsqueda

BST balanceado · $\log_2 n$  **≈ 20 accesos**

Árbol B · t = 500  **3 lecturas**

$n = 1\,000\,000 \cdot \log_2 n \approx 20 \cdot \log_{500} n \approx 2.2$

Altura: **O(log_t n)**, con t de cientos o miles: de **2 a 4 niveles** incluso con millones de registros.

Inserción y split

1

Nodo lleno

La inserción ocurre en una hoja. Con 2t − 1 claves, no cabe otra.

2

Mediana sube

La clave mediana sube al nodo padre.

3

Dos nodos

Las t − 1 claves de cada lado forman un nodo propio.

Split proactivo: se divide todo hijo lleno **antes de bajar** a él. Si la raíz está llena, se crea una nueva raíz: **es la única forma en que el árbol crece en altura.**

t = 2: insertar 10, 20, 30 y 40

Tras insertar 10, 20 y 30

raíz llena (2t − 1 = 3)



Tras insertar 40

nueva raíz



Ahora se inserta **40**: la raíz está llena, hay que hacer **split**.

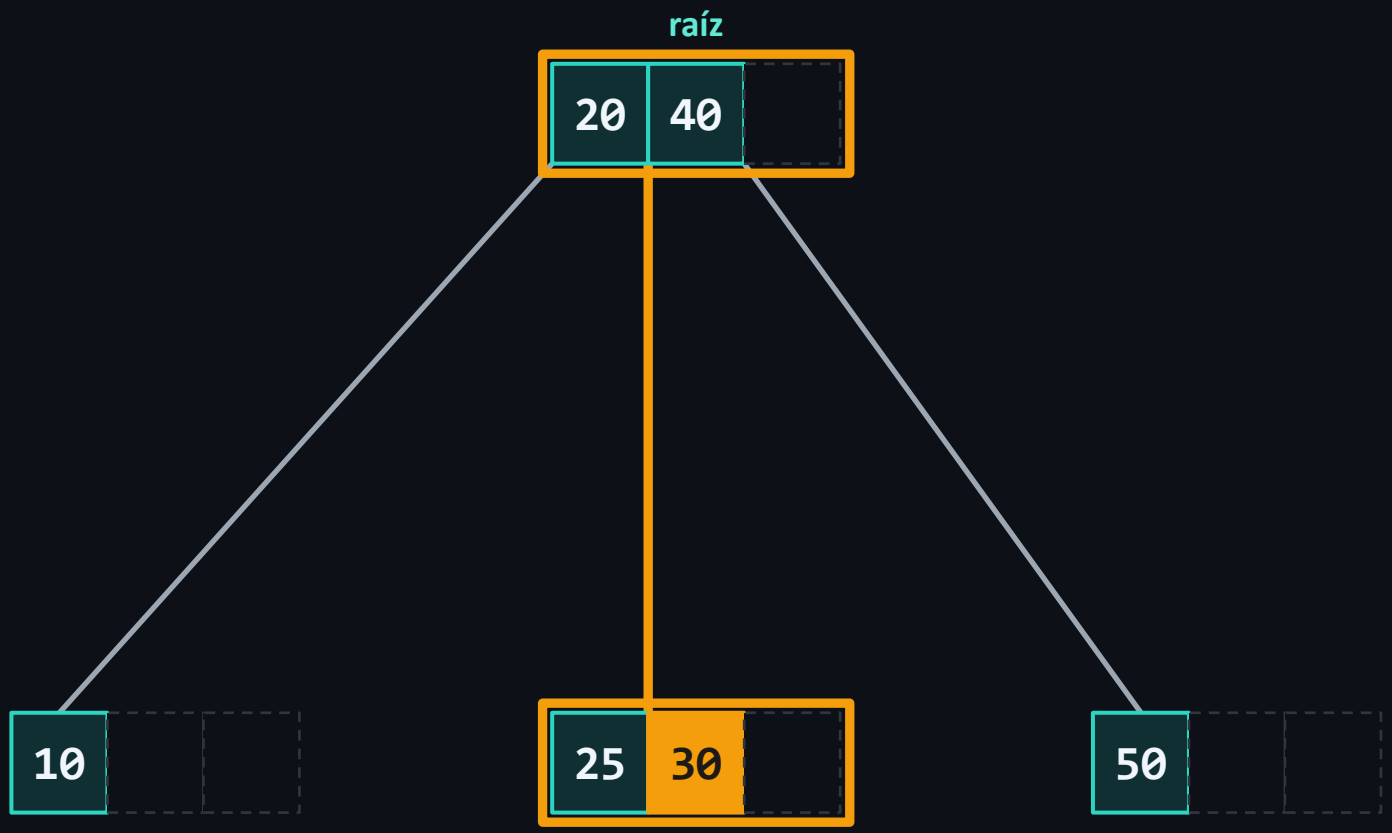
La mediana **20** sube a una nueva raíz y el nodo se parte en [10] y [30]; luego **40** baja al hijo derecho.

t = 2: insertar 50 y 25



25 debe bajar al hijo derecho, que está lleno: **split** proactivo, la mediana **40** sube y 25 entra en [30] porque 20 < 25 < 40.

Resultado y búsqueda de la clave 30



Búsqueda de 30

En la raíz, 20 < 30 < 40: se baja al hijo central, donde está. 2 lecturas.

Balanceado

Todas las hojas están al mismo nivel; el árbol solo creció por la raíz.

Eliminación: conservar el balanceo

Invariante: todo nodo (salvo la raíz) debe conservar al menos t − 1 claves.

Hoja con claves de sobra

Si tiene más de t − 1 claves, basta quitar la clave.

Nodo interno

Se reemplaza por su predecesor o sucesor y se elimina esa clave de la hoja.

Préstamo (borrow)

Con underflow: se toma una clave de un hermano con claves de sobra, pasando por el padre.

Fusión (merge)

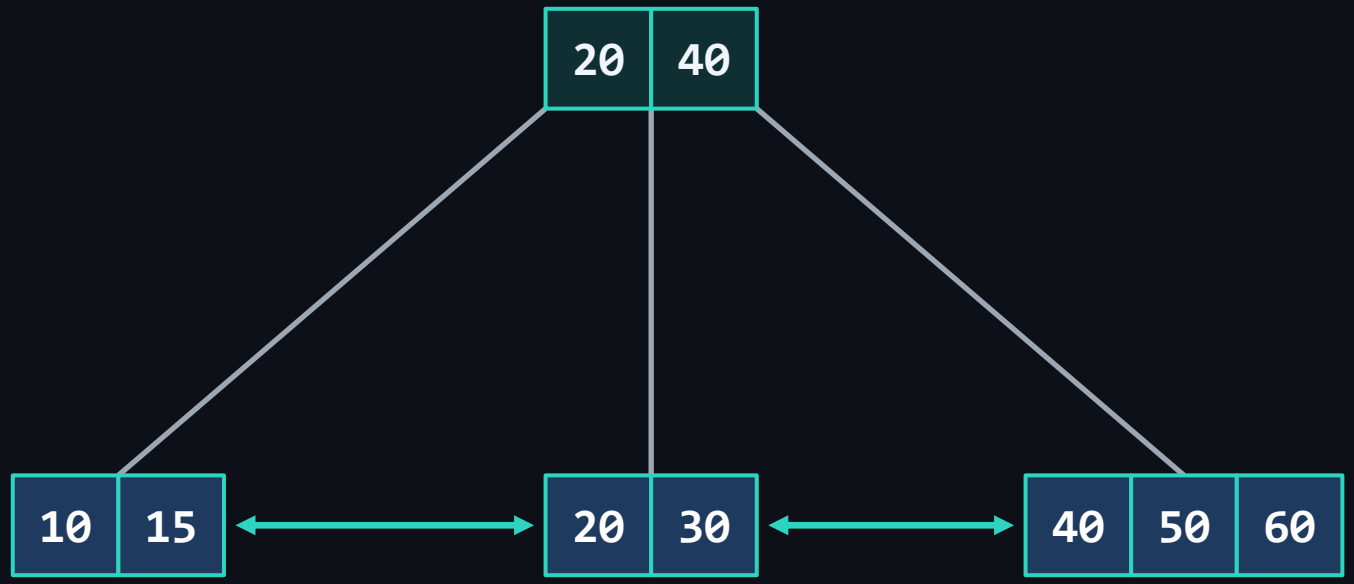
Si ningún hermano puede prestar: se unen dos hermanos con la clave separadora del padre.

// árbol b+

Árbol B+: datos solo en las hojas

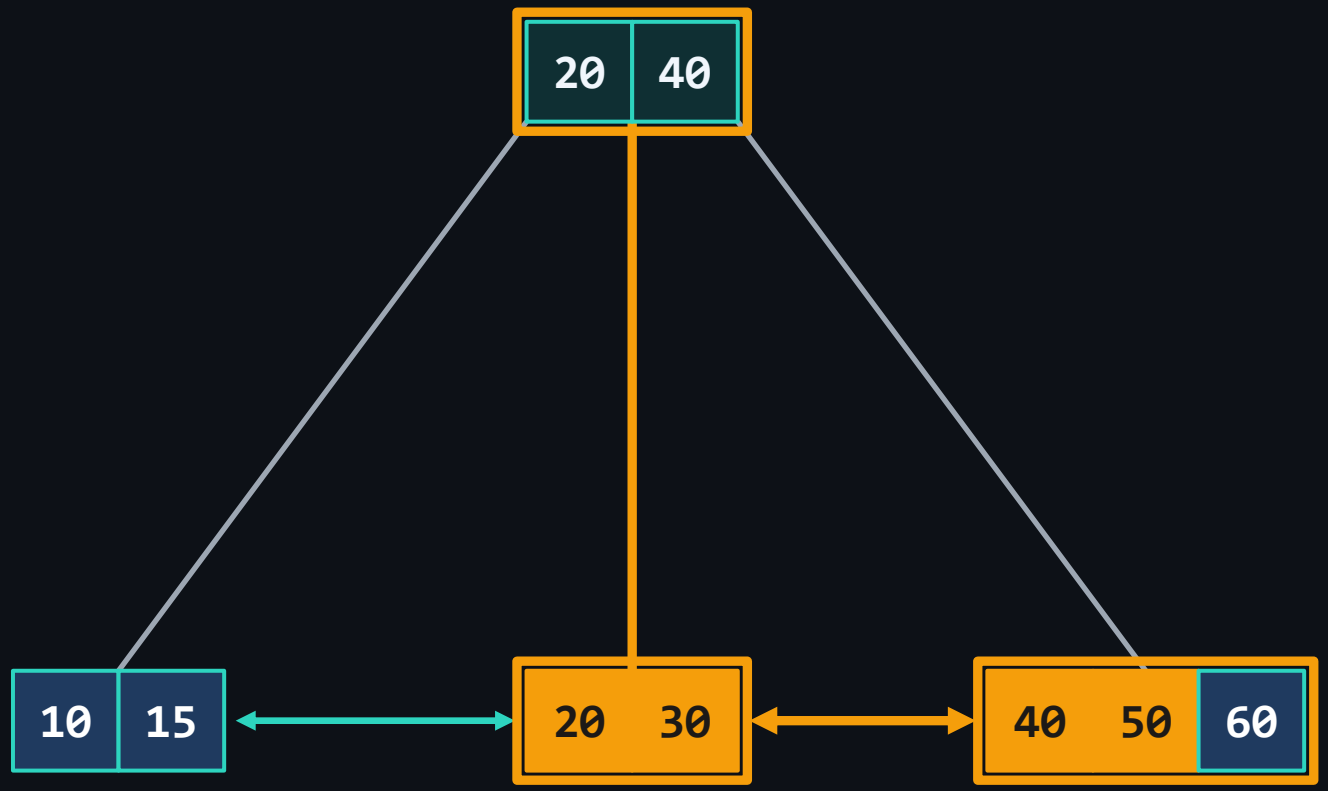
Nodos internos: solo claves de ruteo (copias)

Hojas: los datos, enlazadas entre sí



La clave **20** aparece dos veces: como copia de ruteo en el nodo interno y como dato real en la hoja.

Consulta por rango: claves de 20 a 50



1. Descender

Se busca el límite inferior (20): O(log_t n) accesos hasta su hoja.

2. Recorrer hojas

Se sigue la lista enlazada hasta superar el límite superior (50).

Ventajas del árbol B+

Consultas por rango

Se llega a la hoja inicial y se siguen las hojas enlazadas.

Escaneo secuencial

Recorrer todo en orden es recorrer la lista de hojas; no se vuelve a tocar un nodo interno.

Nodos internos delgados

Solo guardan claves de ruteo: caben más por página, hay más ramificación y menos altura.

Tiempo más uniforme

Los datos siempre están en las hojas: toda búsqueda recorre exactamente la altura del árbol.

// árbol b+

Árbol B vs. árbol B+

Aspecto	Árbol B	Árbol B+
Ubicación de los datos	Nodos internos y hojas	Solo en las hojas
Claves en nodos internos	Datos reales	Solo claves de ruteo (copias)
Hojas enlazadas	No	Sí (lista enlazada)
Rangos y escaneo	Menos eficientes (hay que subir)	Muy eficientes (lista de hojas)
Factor de ramificación	Menor (los nodos guardan datos)	Mayor (solo claves)
Búsqueda exitosa	Puede terminar en un nodo interno	Siempre llega a una hoja

Uso típico. B: propósito general y algunos índices. **B+**: DBMS y sistemas de archivos.

// implementación

Esbozo en C++: el nodo de un árbol B

Nodo con grado mínimo T

```cpp
template <typename Key, int T>
struct BTreeNode {
    // entre T-1 y 2T-1 claves, ordenadas
    std::vector<Key> keys;
    // k claves y no es hoja -> k+1 hijos
    std::vector<std::unique_ptr<BTreeNode>> children;
    bool isLeaf = true;
    bool isFull() const {
        return keys.size() == 2 * T - 1;
    }
};
```

En el disco real

Cada nodo sería una página; aquí hay punteros en memoria.

Nodo lleno

2T − 1 claves: el próximo paso es un split.

> Es un esbozo conceptualmente compilable (C++17); no incluye la inserción ni la eliminación.

// implementación

Esbozo en C++: búsqueda en un árbol B

Busca la clave k a partir de un nodo

```
template <typename Key, int T>
const BTreeNode<Key, T>* search(
        const BTreeNode<Key, T>* node, const Key& k) {
    if (node == nullptr) return nullptr;
    std::size_t i = 0;
    while (i < node->keys.size() && k > node->keys[i]) ++i;
    if (i < node->keys.size() && k == node->keys[i])
        return node;                          // encontrada
    if (node->isLeaf) return nullptr;         // no existe
    return search(node->children[i].get(), k);  // 1 lectura
}
```

Costo

O(log n) comparaciones y **O(log_t n)** accesos a disco.

En un B+

Siempre desciende hasta una hoja.

Cada descenso al hijo `i` es **una lectura de página de disco**. Para nodos grandes, una búsqueda binaria sería preferible al `while`.

// práctica

Ahora usted (1/2): discos y archivos

Resuelva en parejas (las respuestas están en las notas):

A

Capacidad con RAID

Con cuatro discos de 1 TB, ¿qué capacidad útil da RAID 0, RAID 5 y RAID 10?

B

Reconstruir con XOR

El disco 2 falló. En la franja 4 quedan 0110, 1101 y 1010. ¿Qué guardaba el disco 2?

C

Tamaño máximo

Con bloques de 1 KiB y punteros de 4 B (12 directos + 3 indirectos), ¿cuál es el tamaño máximo de un archivo?

// práctica

Ahora usted (2/2): ordenamiento y árboles B

Resuelva en parejas (las respuestas están en las notas):

D

External merge-sort

Un archivo de 8 páginas, runs de 1 página y mezclas de dos en dos: ¿cuántas pasadas y cuántas E/S?

E

Árbol B con t = 2

Al árbol del ejemplo agréguele 60, 70 y 80. Dibuje el árbol resultante.

F

Accesos a disco

Con t = 500 y mil millones de claves, ¿cuántas lecturas necesita una búsqueda? ¿Y en un BST?

// cierre

Resumen

- **Jerarquía de memoria:** más arriba es más rápida, más cara y de menor capacidad
- **HDD vs. SSD:** el disco paga seek + latencia; el SSD no tiene partes móviles
- **RAID:** striping, mirroring y paridad; con XOR se reconstruye un disco perdido
- **Archivos:** abstracción sobre particiones, volúmenes y sistemas de archivos
- **I-nodos:** 12 punteros directos y 3 indirectos ubican los bloques de un archivo
- **Cache:** hit y miss; políticas de reemplazo LRU, FIFO, LFU y MRU
- **Ordenamiento externo:** external merge y merge-sort con runs en disco
- **Árboles B y B+:** un nodo = una página; el B+ guarda los datos en hojas enlazadas

Referencias: GeeksforGeeks (External Sorting; Introduction of B-Tree; B+ Tree) · Koutris, P. CS 564 (Spring 2018), Lecture 16 · Cormen, T. H. et al. Introduction to Algorithms (CLRS), «B-Trees» · valeriodiste.github.io/ExternalMergeSortVisualizer